

Problem Solving Using C (25VBT0C103)



www.onlinejain.com

Module: 05

Industry Use Cases & Project-Based Application

Module Information

Field	Details
Course Title	Problem Solving using C
Course Code	25VBT0C103
Module Number	Module 5 of 5
Module Title	Industry Use Cases and Project-Based Application

Module Overview

Module 5 is the culminating module of this course — where everything you have learned across Modules 1 through 4 converges into industry-ready, project-based application. Rather than studying individual language features in isolation, you now design, build, test, and version-control complete C applications that mirror the kind of software produced by professional developers in the industry.

The module begins by examining the architecture of mini-projects — how a real application is planned before a single line of code is written. You will study project decomposition, module boundaries, and the role of a design document. This mirrors the way development teams at companies like Infosys, TCS, and Wipro start every project engagement.

You will then build two complete menu-driven applications: an ATM Interface System and a Student Grade Management System. Menu-driven programs are a foundational pattern in industry software — from POS terminals at Reliance Fresh to configuration tools on embedded devices. These projects reinforce conditional logic, loops, functions, structs, and file I/O working together in a unified application.

Real-time input handling and validation is the next focus. Real-world programs must be resilient — they cannot crash when a user enters a letter instead of a number. You will learn defensive input techniques using `fgets` and `sscanf`, range validation, and retry loops, which are used in every commercial data-entry application.

Logging systems are a critical feature of production software. Every transaction, error, and event must be recorded for auditing, debugging, and compliance purposes. You will implement a file-based logging system that writes timestamped entries to a persistent log file — the same principle behind the transaction logs at HDFC Bank and the audit trails at Infosys delivery centres.

Structuring code for modularity and readability revisits the header-file and multi-file approach from Module 4, this time in the context of a full project. You will apply naming conventions, function length guidelines, and code organisation principles that make software maintainable over time — a critical quality criterion in long-running IT projects.

Manual testing and test case design teaches you how to verify that your program behaves correctly across normal, boundary, and erroneous inputs. This is followed by Git integration for version tracking — tying together the version control skills from Module 4 into a complete project workflow with meaningful commit history.

The module concludes with a complete capstone project: an Employee Payroll Management System that integrates all the skills from the entire course. Step-by-step guidance, a project structure template, and a submission checklist ensure you can independently deliver a working, well-documented, version-controlled C application.

Learning Objectives

By the end of this module, you will be able to:

- Analyse the architecture of mini-projects and decompose requirements into modular C components.
- Design and build complete menu-driven C applications with validated input, file persistence, and error handling.
- Construct a file-based logging system that records timestamped events for auditing purposes.
- Apply modular programming principles, naming conventions, and code structuring guidelines to produce readable, maintainable C code.
- Create a complete, version-controlled C project with manual test cases and a structured Git commit history.

Learning Outcomes

After completing this module, you will be able to:

- Decompose a project brief into functions, modules, and data structures before writing code.
- Build a working ATM-style and a Student Grade System with persistent binary file storage and comprehensive input validation.
- Implement a reusable `log_event()` function that writes timestamped audit entries to a log file using file I/O.
- Organise a multi-file C project with header files, a Makefile, and meaningful Git commits that document development progress.
- Design a test case table covering normal, boundary, and error scenarios, and verify each case against the expected output.

Table of Topics

Sub-Unit	Topic	Description
5.1	Mini Project Architecture	Requirements analysis, decomposition, design documents, project structure
5.2	Menu-Driven Program: ATM Interface	Account management with file persistence, transactions, balance enquiry
5.3	Menu-Driven Program: Student Grade System	Grade entry, GPA calculation, binary file storage, report generation
5.4	Real-Time Input Handling and Validation	Defensive input using fgets/sscanf, range validation, retry loops
5.5	Logging Systems Using File I/O	Timestamped event logging, log rotation concepts, audit trails
5.6	Modular Code Structure and Readability	Naming conventions, function guidelines, code comments, style rules
5.7	Manual Testing and Test Case Design	Test case tables, boundary value analysis, equivalence partitioning
5.8	Git Integration for Version Tracking	Feature branch workflow, commit conventions, .gitignore, tagging releases
5.9	Capstone Project: Employee Payroll System	Complete project integrating all skills — design, code, test, Git workflow
5.10	Project Submission Checklist	Quality criteria, documentation, final review checklist for submission

5.1 Mini Project Architecture and Design Principles

Before writing a single line of code, professional developers invest time in understanding the problem and designing the solution. This planning stage prevents the most expensive type of mistake in software: building the wrong thing, or building the right thing in a way that cannot be maintained.

A mini-project in C — even one that is 200–500 lines — benefits enormously from upfront design. The principles you learn here scale directly to the enterprise projects undertaken by companies like Infosys, HCL, and Tech Mahindra, where a "project" might span hundreds of thousands of lines across dozens of modules.

Step 1 — Understand the Requirements

Begin by asking: what must the program do? List all features (functional requirements) and constraints (non-functional requirements) explicitly before touching the keyboard.

Example requirements for an ATM System:

- Functional: Create account, deposit, withdraw, check balance, view transaction history, delete account.
- Non-functional: Data must persist across sessions (file storage); invalid input must be gracefully handled; all transactions must be logged.

Step 2 — Decompose into Modules

Decomposition is the art of breaking a large problem into smaller, independently solvable sub-problems. Each sub-problem becomes a module with a clear, single responsibility — a principle known as the Single Responsibility Principle (SRP).

Module	Responsibility	Files
Account Module	CRUD operations on account records	account.h, account.c
Transaction Module	Deposit, withdraw, transfer logic	transaction.h, transaction.c
File I/O Module	Load and save records to binary file	fileio.h, fileio.c
Logger Module	Append timestamped events to log file	logger.h, logger.c
UI Module	Display menus, capture validated input	ui.h, ui.c
Main	Entry point, initialise and orchestrate	main.c

Step 3 — Define Data Structures

Identify the core entities in your application and define C structs for them before writing any functions. This is the most important design decision in a C project — a well-designed struct makes every function cleaner and simpler.


```
/* Core data structure – designed before any function is written
*/

typedef struct {
    int    accountNumber;

    char    holderName[60];

    char    pin[6];          /* Stored as string for simplicity */
    double balance;

    int     isActive;        /* 1 = active, 0 = closed */
} Account;

typedef struct {
    int    accountNumber;

    char    type[10];        /* "DEBIT" or "CREDIT" */
    double amount;

    char    timestamp[20]; /* "YYYY-MM-DD HH:MM:SS" */
} Transaction;
```

Step 4 — Plan the Project Directory Structure

A clean directory layout makes the project navigable and professional:

```
atm_system/
├── include/
│   ├── account.h
│   ├── transaction.h
│   ├── fileio.h
│   ├── logger.h
│   └── ui.h
└── src/
```

```
|   ├── main.c
|   ├── account.c
|   ├── transaction.c
|   ├── fileio.c
|   ├── logger.c
|   └── ui.c
└── data/
    ├── accounts.dat
    ├── transactions.dat
    ├── logs/
    │   └── atm_audit.log
    ├── tests/
    │   └── test_cases.txt
    └── Makefile
```

💡 Did You Know?

The Linux kernel — the world's largest open-source C project with over 30 million lines of code — follows a similar directory structure. Each hardware subsystem (networking, file systems, USB, graphics) has its own subdirectory with dedicated .h and .c files. The modular architecture means a developer can contribute to the USB driver without understanding the file system code. ISRO's NavIC satellite software follows an analogous modular design.

5.2 Menu-Driven Program: ATM Interface System

The ATM Interface is one of the most illustrative menu-driven programs in C education — because it maps directly to software that millions of Indians interact with daily at SBI, HDFC, ICICI, and Axis Bank ATMs. It exercises structs, file I/O, functions, pointers, input validation, and persistent data storage all in one coherent application.

In this sub-unit, you will build a functional ATM simulation with account creation, PIN verification, deposit, withdrawal, balance enquiry, and transaction logging — all backed by binary file persistence.

The Account Struct and File Operations

```
/* File: account.h */

#ifndef ACCOUNT_H
#define ACCOUNT_H

#define MAX_NAME    60
#define DATA_FILE  "data/accounts.dat"
#define MAX_ACCS    100

typedef struct {
    int    accNo;
    char   name[MAX_NAME];
    char   pin[6];
    double balance;
    int    active;
} Account;

int  loadAccounts(Account arr[], int *count);
void saveAccounts(const Account arr[], int count);
int  findAccount(const Account arr[], int count, int accNo);
```

```
void createAccount(Account arr[], int *count);  
void deposit(Account arr[], int count);  
void withdraw(Account arr[], int count);  
void checkBalance(const Account arr[], int count);  
  
#endif
```

Menu Loop — The Heart of the Application

```
/* File: main.c — ATM Interface */  
  
#include <stdio.h>  
#include <stdlib.h>  
#include "account.h"  
#include "logger.h"  
  
void displayMenu() {  
    printf("\n┌───────────────────────────────────┐\n");  
    printf("│      SBI ATM SIMULATION SYSTEM      │\n");  
    printf("│───────────────────────────────────│\n");  
    printf("│  1. Create New Account                │\n");  
    printf("│  2. Deposit                          │\n");  
    printf("│  3. Withdraw                         │\n");  
    printf("│  4. Check Balance                    │\n");  
    printf("│  5. View All Accounts                │\n");  
    printf("│  0. Exit                            │\n");  
    printf("└───────────────────────────────────┘\n");  
    printf("  Enter choice: ");  
}
```

```
int main() {  
    Account accounts[MAX_ACCS];  
    int count = 0;  
    int choice;  
  
    loadAccounts(accounts, &count);  
    log_event("ATM session started.");  
  
    do {  
        displayMenu();  
        scanf("%d", &choice);  
  
        switch (choice) {  
            case 1: createAccount(accounts, &count); break;  
            case 2: deposit(accounts, count);          break;  
            case 3: withdraw(accounts, count);         break;  
            case 4: checkBalance(accounts, count);     break;  
            case 5: listAccounts(accounts, count);     break;  
            case 0: printf("Thank you. Goodbye!\n"); break;  
            default: printf("Invalid option. Try again.\n");  
        }  
        saveAccounts(accounts, count);  
    } while (choice != 0);  
  
    log_event("ATM session ended.");  
    return 0;  
}
```

Deposit Function with Validation

```
void deposit(Account arr[], int count) {  
    int accNo; double amount;  
  
    printf("Enter Account Number: "); scanf("%d", &accNo);  
  
    int idx = findAccount(arr, count, accNo);  
    if (idx == -1) { printf("Account not found.\n"); return; }  
  
    if (!arr[idx].active) { printf("Account is inactive.\n");  
return; }  
  
    printf("Enter deposit amount (Rs.): "); scanf("%lf", &amount);  
    if (amount <= 0) { printf("Amount must be positive.\n");  
return; }  
  
    if (amount > 200000) { printf("Exceeds single-transaction  
limit.\n"); return; }  
  
    arr[idx].balance += amount;  
  
    printf("Deposit successful. New balance: Rs. %.2f\n",  
arr[idx].balance);  
  
    log_event("Deposit of Rs. %.2f to account %d", amount, accNo);  
}
```

🔍 Analysing Misconceptions: A Menu-Driven Program Must Use switch-case

✗ MYTH: You must always use a switch-case statement to implement a menu in C.

✓ REALITY: While switch-case is clean and readable for integer menus, you can also use if-else chains, or even a function pointer array for dynamic dispatch. Many embedded systems use arrays of function pointers (`void (*handlers[])(void)`) for compact, table-driven menu logic. The right choice depends on menu size, extensibility, and performance requirements.

✓ Let's Check

Q.No	Type	Question
1	MCQ	In a menu-driven program, which loop structure is best for repeating the menu until the user chooses to exit?
2	MCQ	What is the purpose of calling <code>saveAccounts()</code> after every menu operation?
3	Fill in Blank	The _____ statement transfers control to a matching constant label within a switch block.
4	Fill in Blank	A _____ guard using <code>#ifndef</code> in a header file prevents duplicate declarations when included in multiple .c files.
5	True/False	In C, a switch-case can be used with char values as the controlling expression.
6	True/False	Without a break statement at the end of a case, execution falls through to the next case.

Answers:

Q.No	Answer
1	b) do-while loop — it guarantees the menu is shown at least once before checking the exit condition.
2	b) To persist data to disk so it is not lost if the program exits.
3	case
4	include
5	True — char is an integer type in C; switch works on any integer expression.
6	True — fall-through is intentional C behaviour; always add break unless fall-through is desired.

5.3 Menu-Driven Program: Student Grade Management System

The Student Grade Management System is another classic real-world application, mirroring the software used by universities and examination boards across India — from autonomous colleges affiliated with Mumbai University to centrally administered systems like the National Board of Examinations (NBE). This project reinforces struct arrays, binary file I/O, GPA computation, and formatted report generation.

Data Structure Design

```
#ifndef STUDENT_H
#define STUDENT_H

#define MAX_STUDENTS 200
#define MAX_SUBJECTS 6
#define STUDENT_FILE "data/students.dat"

typedef struct {
    int    rollNo;
    char   name[60];
    char   course[30];
    int    semester;
    float  marks[MAX_SUBJECTS]; /* Marks for up to 6 subjects */
    float  gpa;
    char   grade;               /* O/A/B/C/D/F */
} Student;

char  computeGrade(float gpa);
float computeGPA(const float marks[], int numSubjects);

void  addStudent(Student arr[], int *count);
void  viewAll(const Student arr[], int count);
void  searchStudent(const Student arr[], int count, int rollNo);
void  generateReport(const Student arr[], int count);

#endif
```

GPA Computation Function

```
/* Compute GPA on 10-point scale (UGC CBCS pattern) */
float computeGPA(const float marks[], int n) {
    float total = 0;
    for (int i = 0; i < n; i++) total += marks[i];
    float avg = total / n;
    /* Map percentage to GPA */
    if (avg >= 90) return 10.0f;
    if (avg >= 80) return 9.0f;
    if (avg >= 70) return 8.0f;
    if (avg >= 60) return 7.0f;
    if (avg >= 50) return 6.0f;
    if (avg >= 40) return 5.0f;
    return 0.0f; /* Fail */
}

char computeGrade(float gpa) {
    if (gpa == 10.0f) return 'O'; /* Outstanding */
    if (gpa >= 9.0f) return 'A';
    if (gpa >= 8.0f) return 'B';
    if (gpa >= 7.0f) return 'C';
    if (gpa >= 6.0f) return 'D';
    if (gpa >= 5.0f) return 'E'; /* Pass */
    return 'F';
}
```

Generating a Formatted Report

A key feature of any grade management system is producing a readable report. The following function prints a formatted class report to both the console and a text file, mimicking the Mark Sheets generated by university systems:

```
void generateReport(const Student arr[], int count) {
    FILE *fp = fopen("reports/result_sheet.txt", "w");
    if (!fp) { printf("Cannot create report file.\n"); return; }

    fprintf(fp, "%-8s %-25s %-6s %-6s %-6s\n",
            "Roll No", "Name", "GPA", "Grade", "Result");
    fprintf(fp, "%s\n", "-----");

    int pass=0, fail=0;
    for (int i = 0; i < count; i++) {
        fprintf(fp, "%-8d %-25s %-6.1f %-6c %s\n",
                arr[i].rollNo, arr[i].name, arr[i].gpa,
                arr[i].grade, arr[i].grade=='F'?"FAIL":"PASS");
        if (arr[i].grade == 'F') fail++; else pass++;
    }

    fprintf(fp, "\nTotal: %d | Pass: %d | Fail: %d\n", count, pass,
    fail);
    fclose(fp);
    printf("Report saved to reports/result_sheet.txt\n");
}
```

5.4 Real-Time Input Handling and Validation

One of the most common causes of bugs, crashes, and security vulnerabilities in C programs is unchecked user input. When a program calls `scanf("%d", &n)` and the user enters "abc", `scanf` fails silently, leaving `n` undefined and input buffer corrupted — leading to infinite loops, wrong calculations, or outright crashes.

Robust input handling is not optional in production software — it is mandatory. Every commercial application, from IRCTC booking forms to Paytm UPI transaction screens, validates every field before processing. In C, this is achieved by reading input as strings first, then parsing and validating before converting.

The Safer Input Pattern: `fgets` + `sscanf`

The gold standard for safe C input is to read the entire line as a string with `fgets`, then parse it with `sscanf`. This keeps the input buffer clean regardless of what the user types.

```
/* Safe integer input with retry loop */
#include <stdio.h>
#include <string.h>

int getValidInt(const char *prompt, int min, int max) {
    char buf[64];
    int value;

    while (1) {
        printf("%s", prompt);
        if (fgets(buf, sizeof(buf), stdin) == NULL) continue;

        /* Remove trailing newline */
        buf[strcspn(buf, "\n")] = '\0';

        if (sscanf(buf, "%d", &value) == 1) {
            if (value < min || value > max) {
                printf("Value out of range. Please enter a valid integer.\n");
                continue;
            }
            return value;
        }
    }
}
```

```
/* Try to parse an integer – sscanf returns 1 on success
*/

if (sscanf(buf, "%d", &value) != 1) {
    printf(" Invalid input. Please enter a number.\n");
    continue;
}

if (value < min || value > max) {
    printf(" Out of range. Enter a value between %d and
%d.\n", min, max);
    continue;
}

return value;
}
}
```

Validating Floating-Point Input

```
double getValidDouble(const char *prompt, double min, double max)
{
    char buf[64]; double value;

    while (1) {
        printf("%s", prompt);
        if (!fgets(buf, sizeof(buf), stdin)) continue;
        buf[strcspn(buf, "\n")] = '\0';
        if (sscanf(buf, "%lf", &value) != 1) {
            printf(" Please enter a valid amount (e.g.,
1500.50).\n");
            continue;
        }

        if (value < min || value > max) {
```

```
        printf("    Amount must be between %.2f and %.2f.\n",
min, max);

        continue;
    }

    return value;
}

}

/* Usage in ATM deposit: */

/* double amt = getValidDouble("Enter deposit amount (Rs.): ", 1.0,
200000.0); */
```

Validating String Input — Names and IDs

```
#include <ctype.h>

/* Validate a name: non-empty, alphabetic characters and spaces
only */
int isValidName(const char *name) {

    if (strlen(name) == 0) return 0;

    for (int i = 0; name[i]; i++) {

        if (!isalpha(name[i]) && name[i] != ' ') return 0;

    }

    return 1;
}

/* Get validated name from user */
void getValidName(const char *prompt, char *dest, int maxLen) {

    char buf[100];

    while (1) {
```

```
printf("%s", prompt);  
fgets(buf, sizeof(buf), stdin);  
buf[strcspn(buf, "\n")] = '\0';  
if (isValidName(buf)) {  
    strncpy(dest, buf, maxLen - 1);  
    dest[maxLen-1] = '\0';  
    return;  
}  
printf("  Invalid name. Use letters and spaces only.\n");  
}  
}
```

🔍 Analysing Misconceptions: scanf Is Sufficient for User Input

✗ MYTH: Using `scanf("%d", &n)` is the standard and safe way to read integers from the user.

✓ REALITY: `scanf` leaves unread characters in the input buffer when it fails, causing subsequent reads to behave unexpectedly. It also cannot detect when the user enters a number out of valid range. The professional pattern is to always read the full line with `fgets` into a buffer, then parse and validate using `sscanf`. This pattern is used in command-line tools, embedded configuration parsers, and server applications written in C.

5.5 Logging Systems Using File I/O

A logging system is one of the most valuable components you can add to any application. Logs are the audit trail that allows developers and administrators to understand exactly what the program did, when it did it, and what input triggered each action. They are indispensable for debugging, security auditing, and regulatory compliance.

Every major enterprise system in India generates logs. HDFC Bank's core banking system logs every transaction with microsecond timestamps. Infosys's delivery management systems log every user action for quality auditing. Even IRCTC's ticketing engine logs every booking attempt for fraud detection and capacity planning.

The Logger Module

A well-designed logger is implemented as a standalone module with a single public function: `log_event()`. This function accepts a format string (like `printf`), prepends a timestamp, and appends the entry to a persistent log file.

```
/* File: logger.h */

#ifndef LOGGER_H
#define LOGGER_H

#define LOG_FILE "logs/app_audit.log"

/* Log a formatted event — usage: log_event("Deposit of Rs. %.2f",
amount) */
void log_event(const char *format, ...);

#endif
```

```
/* File: logger.c */

#include <stdio.h>
```



```
#include <stdarg.h>

#include <time.h>

#include "logger.h"

void log_event(const char *format, ...) {

    FILE *fp = fopen(LOG_FILE, "a"); /* Append mode */

    if (!fp) return;

    /* Get current timestamp */

    time_t now = time(NULL);

    struct tm *t = localtime(&now);

    char timestamp[25];

    strftime(timestamp, sizeof(timestamp), "%Y-%m-%d %H:%M:%S",

t);

    /* Write timestamp prefix */

    fprintf(fp, "[%s] ", timestamp);

    /* Write the formatted message using variadic args */

    va_list args;

    va_start(args, format);

    vfprintf(fp, format, args);

    va_end(args);

    fprintf(fp, "\n");

    fclose(fp);

}
```

Sample Log Output

[2024-03-15 09:14:02] ATM session started.
[2024-03-15 09:14:18] Account 100201 created for Priya Nair.
[2024-03-15 09:15:03] Deposit of Rs. 25000.00 to account 100201.
[2024-03-15 09:16:45] Withdrawal of Rs. 5000.00 from account 100201.
[2024-03-15 09:17:02] Failed withdrawal: Insufficient funds. Account 100203.
[2024-03-15 09:17:33] ATM session ended.

Log Levels

Production logging systems differentiate entries by severity using log levels. You can extend the logger with a level parameter:

Level	When to Use	Example
INFO	Normal operational events	Session started, deposit successful
WARN	Non-critical problems	Invalid login attempt, low balance
ERROR	Failures that affect a single operation	File open failure, record not found
CRITICAL	System-wide failures	Database corruption, file system full

 Let's Check

Q.No	Type	Question
1	MCQ	Which file open mode should be used when writing log entries to ensure existing entries are not deleted?
2	MCQ	Which C library function is used to get the current date and time in log_event()?
3	Fill in Blank	The _____ function formats a string with a variable argument list, like printf but writing to a FILE *.
4	Fill in Blank	strftime() converts a struct tm * time value into a _____ format string.
5	True/False	Opening a log file in "w" mode each time log_event() is called would preserve existing log entries.
6	True/False	The <stdarg.h> header provides macros (va_list, va_start, va_end) needed for variadic functions.

Answers:

Q.No	Answer
1	b) "a" (append mode) — adds new entries at the end without truncating the file.
2	b) time(NULL) from <time.h>
3	vfprintf
4	human-readable formatted

5	False — "w" mode truncates (clears) the file on every open. Use "a" (append) for logs.
6	True — <stdarg.h> is required for implementing variadic functions like log_event().

5.6 Modular Code Structure and Readability

The quality of code is not measured solely by whether it produces correct output. In industry, code is read far more often than it is written — by teammates, code reviewers, future maintainers, and automated analysis tools. Code that is difficult to understand is expensive to maintain, risky to modify, and brittle in production.

At TCS, Infosys, and Wipro, developers are evaluated not only on functionality but also on code quality: readability, naming clarity, function design, and adherence to project coding standards. These are not cosmetic concerns — they directly affect project cost, defect rates, and developer velocity.

Naming Conventions

Entity	Convention	Example
Functions	Verb + Noun, camelCase	calculateGPA(), loadAccounts(), validateInput()
Variables	Noun or adjective+noun, camelCase	studentCount, isValid, totalMarks
Constants / Macros	ALL_CAPS with underscore	MAX_STUDENTS, RATE_PER_UNIT, LOG_FILE
Struct types	PascalCase via typedef	Account, Student, Transaction

File names	lowercase with underscore	account.c, logger.h, file_io.c
Booleans	is / has / can prefix	isActive, hasPin, canWithdraw

Function Design Guidelines

- Single Responsibility — every function should do exactly one thing. If you find yourself naming a function `processAndSaveAndLog()`, it should be three functions.
- Length — aim for functions under 40 lines. If a function is longer, it is likely doing too much and can be decomposed.
- Return early — validate preconditions at the start and return immediately on failure. This avoids deeply nested if-else chains.
- Avoid global state — pass data through function parameters and return values rather than modifying global variables.
- Descriptive parameters — `int n` is poor; `int studentCount` is clear.

Commenting Best Practices

Comments should explain WHY, not WHAT. Code should be written clearly enough that what it does is obvious; comments clarify intent, reasoning, and non-obvious behaviour.

```
/* Poor comment — restates what the code obviously does */
i = i + 1; /* Add 1 to i */

/* Good comment — explains why */
/* Skip the header row (row 0) — it contains column labels, not
data */

for (int i = 1; i < rowCount; i++) {
    processRow(data[i]);
}
```

```
}

/* Good function-level comment — explains contract */
/*
 * findAccount: Searches the accounts array for a matching account
number.
 * Returns the index of the account if found, or -1 if not found.
 * The array is assumed to be unordered (linear search).
 */
int findAccount(const Account arr[], int count, int accNo) { ... }
```

💡 Did You Know?

Google's C++ Style Guide — widely adopted by Indian tech companies like Flipkart, Ola, and PhonePe for their C/C++ services — specifies that functions should have names that are "full words" and should fit in the visible screen. Google internally enforces code style via an automated tool called clang-format, which formats code consistently across millions of lines. Many Indian IT companies have adopted similar automation using tools like AStyle and uncrustify for C codebases.

5.7 Manual Testing and Test Case Design

Writing code is only half the job — verifying that the code is correct is equally important. Testing is the systematic process of exercising a program with specific inputs and checking that the outputs match expectations. In professional development, untested code is not considered complete.

Manual testing — the approach covered here — involves designing test cases, running the program with those inputs, and recording the actual output. Even as automated testing frameworks become common, manual testing remains important for exploratory testing and final validation.

What Makes a Good Test Case?

A test case consists of four components:

- Test ID — a unique identifier (e.g., TC-001).
- Input — the specific input value(s) or sequence of actions.
- Expected Output — what the program should produce given that input.
- Actual Output — what the program actually produces (recorded during testing).

Categories of Test Cases

Category	Purpose	Example (ATM Deposit)
Normal Happy Path	Verify correct behaviour with valid input	Deposit Rs. 5,000 to account with balance Rs. 10,000 → Balance = Rs. 15,000
Boundary Values	Test at the exact limits of accepted ranges	Deposit Rs. 1 (minimum) → Accept; Deposit Rs. 200,000 (maximum) → Accept
Edge Cases	Inputs just outside valid range	Deposit Rs. 0 → Reject; Deposit Rs. 200,001 → Reject

Error / Negative	Invalid or malformed input	Deposit "abc" as amount → Prompt re-entry; Account 9999 (nonexistent) → Error
Stress	Repeated or extreme operations	Create 100 accounts back-to-back → All stored correctly

Sample Test Case Table: ATM Withdrawal Function

TC-ID	Category	Input	Expected Output	Pass/Fail
TC-01	Normal	Acc: 1001, Amount: 2000 (Bal=10000)	New balance: Rs. 8000.00	
TC-02	Boundary	Acc: 1001, Amount: 10000 (full balance)	Balance: Rs. 0.00	
TC-03	Edge	Acc: 1001, Amount: 10001 (Bal=10000)	Error: Insufficient funds	
TC-04	Edge	Amount: 0	Error: Amount must be positive	
TC-05	Error	Acc: 9999 (nonexistent)	Error: Account not found	
TC-06	Error	Amount: "xyz"	Prompt re-entry	

Run each test case manually, record the actual output in the Pass/Fail column, and investigate any cases where actual output does not match expected output. These discrepancies are bugs — document them and fix them before submission.

Analysing Misconceptions: Testing Is Done After Coding

✗ MYTH: You should finish all the code first, then test it at the end.

✓ REALITY: Professional development practices recommend writing test cases alongside or even before code. Test cases derived from requirements help clarify exactly what each function must do. When a function is small and well-defined, its test cases can be written before implementation — a technique known as Test-Driven Development (TDD). Even without formal TDD, designing test cases early reveals ambiguities in requirements before they become bugs in code.

5.8 Git Integration for Version Tracking in Projects

In Module 4, you learned the individual Git commands — init, add, commit, push, branch, merge. In this sub-unit, you apply those commands to a real project with discipline and purpose. A good Git history is not just a backup — it is a narrative of the project's evolution that allows any team member to understand what changed, why it changed, and who changed it.

The .gitignore File

Not all files in a project should be tracked by Git. Compiled binaries, object files, log files, and editor configuration files should be excluded. Create a .gitignore in the project root:

```
# .gitignore for a C project

# Compiled output

*.o

*.out

*.exe

build/

# Binary data files (generated at runtime)
data/*.dat

# Log files (runtime-generated)
logs/*.log

# Editor/IDE metadata
.vscode/
.idea/
*.swp

# OS metadata
.DS_Store
Thumbs.db
```

Conventional Commit Messages

Clear, structured commit messages make the project history readable. A widely adopted convention is:

```
type(scope): short description (max 72 characters)
```

```
# Types:
```

```
# feat      - a new feature
```

```
# fix       - a bug fix
```

```
# docs      - documentation changes only
```

```
# refactor  - code restructuring without behaviour change
```

```
# test      - adding test cases
```

```
# chore     - build/config changes (Makefile, .gitignore)
```

```
# Examples:
```

```
git commit -m "feat(account): add account creation with binary file  
persistence"
```

```
git commit -m "fix(withdraw): reject withdrawal when amount exceeds  
balance"
```

```
git commit -m "test: add test case table for ATM deposit and  
withdraw"
```

```
git commit -m "chore: add .gitignore and Makefile for ATM project"
```

A Git Workflow for a Mini Project

```
# 1. Initialise repository and make first commit
```

```
git init
```

```
git add .gitignore Makefile include/ src/main.c
```

```
git commit -m "chore: project scaffold with Makefile and directory  
structure"
```

```
# 2. Develop account module on a feature branch
git checkout -b feature/account-module

# ... write account.h and account.c ...
git add include/account.h src/account.c

git commit -m "feat(account): implement Account struct and binary
file I/O"

# 3. Add input validation
git add src/ui.c include/ui.h

git commit -m "feat(ui): add validated integer and float input
functions"

# 4. Merge feature branch into main
git checkout main
git merge feature/account-module

# 5. Tag a stable release
git tag -a v1.0 -m "Version 1.0 - ATM core features complete"
git push origin main --tags
```

Release Tagging

Git tags mark specific commits as significant milestones — typically releases. Unlike branches, tags do not move; they permanently label a commit. This is important for traceability: when a customer reports a bug, you can check out the exact tagged version they were using.

💡 Did You Know?

India's largest ride-hailing platform, Ola, manages its backend C/C++ services using Git with a trunk-based development model — all developers commit to the main branch via short-lived feature branches. Every deployment is tagged with a semantic version (v2.4.1 etc.), enabling precise rollback within minutes if a release causes issues. This approach is also followed by Zepto for their inventory management microservices written in C++.

5.9 Capstone Project: Employee Payroll Management System

The Employee Payroll Management System is the flagship capstone project for this course. It integrates every skill covered across all five modules: structs and arrays, file I/O (binary and text), pointers, dynamic memory, input validation, modular design with header files, a Makefile build system, a logging system, and Git version tracking.

This project mirrors real-world payroll software used by Indian companies. TCS processes payroll for over 600,000 employees using automated systems; smaller companies often use custom C-based payroll tools for specialised industries such as manufacturing, construction, and hospitality.

Project Requirements

- Store and manage employee records (up to 500 employees) with binary file persistence.
- Calculate gross salary, deductions (PF, ESI, TDS), and net salary automatically.
- Generate individual payslips and a monthly payroll report to a text file.
- Provide a full menu interface with validated input.
- Log every operation (add, update, delete, generate report) with timestamps.
- Organise code into at least 4 modules: employee, payroll, fileio, logger.

- Build using a Makefile; track using Git with at least 8 meaningful commits.

Data Structures

```
/* employee.h */

#ifndef EMPLOYEE_H
#define EMPLOYEE_H

#define MAX_EMP      500
#define EMP_FILE     "data/employees.dat"

typedef struct {
    int    empId;
    char   name[60];
    char   department[40];
    char   designation[40];
    double basicSalary;
    int    isActive;
} Employee;

typedef struct {
    int    empId;
    int    month;          /* 1-12 */
    int    year;

    double basicSalary;
    double hra;            /* House Rent Allowance = 40% of basic
    */
    double da;             /* Dearness Allowance = 20% of basic */
    double grossSalary;
    double pf;             /* Provident Fund = 12% of basic */
```

```
double esi;          /* ESI = 0.75% of gross (if gross <=
21000) */

double tds;          /* TDS – simplified: 10% if gross > 50000
*/

double netSalary;

} PaySlip;

#endif
```

Payroll Calculation Function

```
/* payroll.c – Calculate payslip for an employee */
#include "employee.h"
#include "payroll.h"

PaySlip calculatePaySlip(const Employee *e, int month, int year) {

    PaySlip ps;

    ps.empId      = e->empId;

    ps.month      = month;

    ps.year       = year;

    ps.basicSalary = e->basicSalary;

    ps.hra   = e->basicSalary * 0.40;
    ps.da    = e->basicSalary * 0.20;
    ps.grossSalary = ps.basicSalary + ps.hra + ps.da;

    ps.pf   = ps.basicSalary * 0.12;

    ps.esi = (ps.grossSalary <= 21000) ? ps.grossSalary * 0.0075 :
0.0;
```

```

    ps.tds = (ps.grossSalary > 50000) ? ps.grossSalary * 0.10 :
0.0;

    ps.netSalary = ps.grossSalary - ps.pf - ps.esi - ps.tds;

    return ps;

}

```

Payslip Print Function

```

void printPaySlip(const Employee *e, const PaySlip *ps, FILE *fp)
{
    fprintf(fp,
"\n===== \n");
    fprintf(fp, "||      EMPLOYEE PAY SLIP      || \n");
    fprintf(fp,
"||=====|| \n");
    fprintf(fp, "|| Emp ID      : %-30d|| \n", ps->empId);
    fprintf(fp, "|| Name        : %-30s|| \n", e->name);
    fprintf(fp, "|| Department: %-30s|| \n", e->department);
    fprintf(fp, "|| Month       : %02d/%04d      || \n",
ps->month, ps->year);
    fprintf(fp,
"||=====|| \n");
    fprintf(fp, "|| Basic Salary      : Rs. %12.2f      || \n", ps-
>basicSalary);
    fprintf(fp, "|| HRA (40%%)        : Rs. %12.2f      || \n", ps-
>hra);
    fprintf(fp, "|| DA (20%%)         : Rs. %12.2f      || \n", ps-
>da);
    fprintf(fp, "|| GROSS SALARY      : Rs. %12.2f      || \n", ps-
>grossSalary);
}

```



```

        fprintf(fp,
"=====\\n");

        fprintf(fp, "PF (12%%) : Rs. %12.2f \\n", ps-
>pf);

        fprintf(fp, "ESI (0.75%%) : Rs. %12.2f \\n", ps-
>esi);

        fprintf(fp, "TDS (10%%) : Rs. %12.2f \\n", ps-
>tds);

        fprintf(fp, "NET SALARY : Rs. %12.2f \\n", ps-
>netSalary);

        fprintf(fp,
"=====\\n");
}

```

The Makefile

```

# Makefile - Employee Payroll Management System

CC      = gcc
CFLAGS  = -Wall -Wextra -g -std=c11
SRC      = src/main.c src/employee.c src/payroll.c src/fileio.c
src/logger.c
OBJ      = $(SRC:.c=.o)
TARGET  = payroll_system

all: $(TARGET)

$(TARGET): $(OBJ)

$(CC) $(CFLAGS) $(OBJ) -o $(TARGET)

%.o: %.c

```

<code>\$(CC) \$(CFLAGS) -Iinclude -c \$< -o \$@</code>
<code>.PHONY: clean run</code>
<code>clean:</code>
<code>rm -f src/*.o \$(TARGET)</code>
<code>run: \$(TARGET)</code>
<code>./\$(TARGET)</code>

5.10 Project Submission Checklist

Before submitting any C project — whether for a course assignment, an internship task, or a professional delivery — conduct a systematic review against this checklist. Professional developers call this a Definition of Done (DoD): a set of agreed criteria that code must satisfy before it is considered complete.

Code Quality Checklist

✓	Criterion
☐	Code compiles with <code>gcc -Wall -Wextra -std=c11</code> with zero warnings.
☐	All functions are under 40 lines; long functions are decomposed into helpers.
☐	Meaningful, consistent names used for all variables, functions, and constants.
☐	No magic numbers — all constants are defined with <code>#define</code> or <code>const</code> .
☐	All files opened with <code>fopen()</code> have a NULL check and are closed with <code>fclose()</code> .
☐	User input is validated — invalid input causes a clear error message and retry.
☐	All <code>malloc/calloc</code> allocations are freed; no memory leaks (verified with Valgrind if available).

☐	Each .c file has a corresponding .h file with include guards.
☐	Functions are documented with a brief comment explaining their contract.
☐	The project builds with a single make command using the provided Makefile.

Testing Checklist

✓	Criterion
☐	A test case table covers: normal inputs, boundary values, edge cases, and error inputs.
☐	All test cases have been manually executed and actual output recorded.
☐	All test cases pass (actual output matches expected output).
☐	Error paths tested: invalid account, negative amount, non-numeric input.
☐	File persistence tested: data survives program exit and restart.

Git and Documentation Checklist

✓	Criterion
☐	Repository initialised with git init; .gitignore excludes *.o, *.dat, *.log, binaries.
☐	At least 8 meaningful commits with conventional commit messages (feat/fix/chore/test).
☐	Each feature or module was committed separately (not everything in one giant commit).
☐	README.md present with: project description, how to build, how to run, and sample output.

-
- | | |
|--------------------------|---|
| ☐ | A release tag (v1.0) created with <code>git tag -a v1.0 -m "description"</code> . |
|--------------------------|---|

💡 Did You Know?

At Amazon India and Flipkart's engineering teams, every code change goes through a mandatory "ready for review" checklist before a Pull Request can be submitted. This includes: zero linter warnings, test coverage for new functions, updated documentation, and a clean build in CI. This culture of quality gates ensures that the main codebase always remains in a deployable state — a discipline that starts with habits formed as a student.

■ Case Study: Infosys BPM — C-Based Payroll Verification Engine for a Manufacturing Client

Background:

Infosys BPM (Business Process Management) manages HR and payroll operations for a large Indian automobile components manufacturer with 12,000 workers across 4 plants in Pune, Chennai, Manesar, and Ahmedabad. The payroll system processes monthly salary for shop-floor workers, whose salary components include variable overtime, attendance-linked allowances, and statutory deductions under the Employees' Provident Fund Organisation (EPFO) and ESI Corporation (ESIC) rules. (Based on Infosys BPM manufacturing payroll domain documentation, 2022.)

Problem:

The legacy payroll engine was a monolithic C program of ~8,000 lines in a single file. It had no input validation, no logging, and no modular separation. Monthly payroll runs would occasionally produce incorrect net salary figures for workers with variable attendance. When discrepancies were found, it was impossible to audit which input caused the error — there was no log. Fixing a single calculation bug required understanding the entire 8,000-line file, and changes routinely introduced new defects. The company escalated to Infosys BPM for a complete rearchitecture.

Solution:

The Infosys team rebuilt the payroll engine as a modular C project with the following architecture:

- `employee.h` / `employee.c` — Employee record CRUD with binary file persistence (`fread/fwrite`), supporting 12,000+ records with $O(n)$ search.
- `payroll.h` / `payroll.c` — All salary calculations isolated in pure functions taking an `Employee *` and returning a `PaySlip struct`. Each calculation rule (PF: 12%,

ESI: 0.75%, LWF: statutory) is defined as a `#define` constant for easy compliance updates.

- `validator.h / validator.c` — All input (employee IDs, attendance days, overtime hours) validated using the `fgets + sscanf` pattern with range checks before entering the computation pipeline.
- `logger.h / logger.c` — Every payroll run, every record update, and every error is logged to a timestamped audit file using the variadic `log_event()` pattern. Log entries include: employee ID, operation, before/after values.
- A Makefile with separate debug (`CFLAGS=-g -Wall`) and release (`CFLAGS=-O2`) targets; builds the entire project in under 10 seconds.
- Git repository with feature branches per pay period; each monthly payroll run is tagged (payroll-2024-03, etc.) for traceability.

Results:

After the rearchitecture: Payroll calculation errors fell to zero in the first three months of operation. Audit investigations that previously took 2–3 days (manually reading console output) now take under 15 minutes using the structured log files. When EPFO revised PF calculation rules in January 2023, updating the single `PF_RATE` `#define` constant and recompiling took under 5 minutes, compared to hours of surgery on the legacy monolith. The project was recognised as a best-practice modular C engineering case study within Infosys BPM's manufacturing vertical.

Key C Concepts Demonstrated:

- Modular C architecture — multiple `.h/.c` files, each with a single responsibility
- Struct-based data modelling — Employee and PaySlip structs separating data from logic
- Binary file I/O — `fread/fwrite` for 12,000-employee record persistence
- Validated input — `fgets + sscanf` pattern preventing erroneous data entry

- Variadic logging — `log_event()` with timestamps for auditability
- Makefile build — debug and release targets, pattern rules for all `.c` files
- Git version control — feature branches, conventional commits, release tagging

Module Summary

Module 5 has brought together the full breadth of your C programming journey into project-based, industry-ready application. You have moved beyond writing individual functions and programs to designing, building, testing, and deploying complete software systems that mirror real-world professional practice.

You began by learning how professional developers approach projects before writing code — through requirements analysis, module decomposition, data structure design, and directory planning. This upfront investment prevents the most costly bugs: architectural ones that are expensive to fix late in development.

The two menu-driven projects — the ATM Interface System and the Student Grade Management System — gave you hands-on experience building cohesive applications that integrate structs, file I/O, validation, and formatted output. These are the class of programs that run on POS systems, kiosk terminals, and administrative workstations across India.

Key takeaways from this module:

- Decompose every project into modules with single responsibilities before writing code — this is the most important habit to build as a developer.
- Always validate input using the `fgets + sscanf + range check` pattern — never trust raw `scanf` in production code.
- Add a logger module to every project that stores data or performs transactions — logs are the only reliable audit trail when something goes wrong.
- Write code for readability: meaningful names, short functions, WHY-focused comments, and consistent style.
- Design test cases across normal, boundary, and error categories before running the program — a test table is a professional deliverable, not an afterthought.
- Maintain a clean Git history with conventional commit messages — your commit log tells the story of the project to every future maintainer.

-
- Use a Makefile to automate the entire build — running make should be the only command needed to compile the project from scratch.
 - Apply the submission checklist before every delivery — zero warnings, validated input, persistent data, meaningful tests, clean Git history.

The Employee Payroll Management System capstone project demonstrates that all of these elements — structs, binary I/O, validation, logging, modularity, Makefile, and Git — can work together in a single, coherent, production-quality C application. The skills you have developed across this course form the foundation of embedded systems programming, systems software development, and the C/C++ backend work that powers India's technology ecosystem.

Glossary

Term	Definition
Menu-Driven Program	A program that displays a list of numbered options and executes the corresponding function based on user selection, typically in a loop until the user exits.
Input Validation	The process of checking user input against rules (type, range, format) before using it in computation — essential for robust, crash-resistant programs.
fgets + sscanf Pattern	A safe C input technique: read the full input line as a string with fgets, then parse and validate using sscanf, avoiding input buffer corruption from invalid entries.
Logger Module	A reusable module providing a log_event() function that writes timestamped entries to a persistent log file, supporting audit, debugging, and compliance requirements.
Variadic Function	A function that accepts a variable number of arguments, declared with ... and accessed using va_list, va_start, va_arg, and va_end macros from <stdarg.h>.
Modular Programming	The practice of dividing a program into independent, interchangeable modules, each with a single responsibility, implemented as separate .h and .c file pairs.
Test Case	A documented specification of an input, the expected output, and the actual output used to verify that a specific behaviour of a program is correct.

Boundary Value Analysis	A testing technique that designs test cases at the exact limits of valid input ranges (minimum, maximum, and just outside both) where defects most commonly occur.
.gitignore	A text file in a Git repository root that lists file patterns Git should not track — typically compiled binaries, object files, log files, and editor metadata.
Definition of Done	An agreed-upon checklist of quality criteria that a piece of software must satisfy before it is considered complete and ready for delivery or submission.

Self Assessment Questions

Section A: Multiple Choice Questions (MCQ)

1. What is the first step a professional developer takes before writing code for a new project?
 - a) Write the main function
 - b) Analyse requirements and design module structure
 - c) Create the Makefile
 - d) Initialise the Git repository
2. In a menu-driven program, which loop guarantees the menu is displayed at least once before the exit condition is checked?
 - a) while
 - b) for
 - c) do-while
 - d) switch
3. What is the correct return value of `findAccount()` when the account is not found in the array?
 - a) 0
 - b) NULL
 - c) -1
 - d) EOF
4. Which function is used in the `fgets + sscanf` input validation pattern to read the full input line?
 - a) `scanf()`
 - b) `gets()`
 - c) `fgets()`
 - d) `readline()`

-
5. What is the role of `sscanf()` in the validated input pattern?
- a) Read from a file
 - b) Parse the string buffer into a typed value
 - c) Display the prompt
 - d) Clear the input buffer
6. Which `<time.h>` function converts a `struct tm *` to a formatted string like "2024-03-15 09:14:02"?
- a) `time()`
 - b) `ctime()`
 - c) `strftime()`
 - d) `localtime()`
7. In the logger module, why is the log file opened in "a" (append) mode rather than "w" mode?
- a) Append mode is faster
 - b) "w" mode is not supported on all systems
 - c) Append mode preserves existing log entries
 - d) Append mode creates the file if absent, "w" does not
8. Which header provides `va_list`, `va_start`, and `va_end` for implementing variadic functions?
- a) `<stdio.h>`
 - b) `<stdlib.h>`
 - c) `<string.h>`
 - d) `<stdarg.h>`
9. What does Single Responsibility Principle (SRP) mean in modular C programming?
- a) Each .c file must be under 100 lines
 - b) Each module/function should have one clearly defined responsibility
 - c) The `main()` function handles all input
-

d) One struct per program

10. Which test case category specifically targets inputs at the exact limits of the valid input range?

- a) Normal / Happy Path
- b) Stress Test
- c) Boundary Value
- d) Error Test

11. In the Employee PaySlip struct, how is HRA (House Rent Allowance) typically calculated?

- a) 12% of basic salary
- b) 40% of basic salary
- c) 0.75% of gross salary
- d) 20% of gross salary

12. What does the .gitignore file accomplish in a C project repository?

- a) Prevents commits to main branch
- b) Lists files and patterns that Git should not track
- c) Encrypts sensitive source files
- d) Deletes compiled binaries on each commit

13. Which conventional commit type prefix should be used for a bug fix?

- a) feat
- b) refactor
- c) fix
- d) chore

14. What is the purpose of `git tag -a v1.0 -m "description"` in a project workflow?

- a) Creates a new branch
- b) Permanently labels a specific commit as a release version

- c) Pushes all commits to remote
- d) Merges the working branch into main

15. Which gcc flag generates debug symbol information required for GDB to map machine code to source lines?

- a) -O2
- b) -Wall
- c) -g
- d) -std=c11

16. What should a README.md file in a C project typically contain?

- a) Only the license text
- b) Project description, build instructions, run instructions, and sample output
- c) The complete source code
- d) Only the test case table

17. In the payroll calculation, which deduction applies only when gross salary is Rs. 21,000 or below?

- a) TDS
- b) PF
- c) HRA
- d) ESI

18. Which Makefile automatic variable expands to the name of the current rule's target?

- a) \$<
- b) \$^
- c) \$@
- d) \$*

19. What is the purpose of the Definition of Done (DoD) checklist before project submission?

- a) To estimate project cost
- b) To ensure all agreed quality criteria are met before delivery
- c) To count lines of code
- d) To select the correct compiler

20. Which C function from `<string.h>` returns the position of the first newline in a string — used to strip the newline after `fgets`?

- a) `strlen()`
- b) `strchr()`
- c) `strcspn()`
- d) `strcat()`

Section B: True / False

State whether each statement is True or False.

1. A menu-driven program should use `exit()` inside each case to terminate when the user makes a selection.
2. The `fgets + sscanf` pattern is safer than `scanf` because it prevents input buffer corruption when the user enters unexpected data types.
3. Using "a" (append) mode in `fopen()` will create the log file if it does not already exist.
4. In the payroll system, the TDS deduction applies to all employees regardless of gross salary.
5. A `.gitignore` file can use wildcard patterns such as `*.o` and `*.dat` to exclude multiple files with the same extension.

Section C: Short Answer Questions

Answer each question in 4–8 sentences or with relevant code and tables.

1. Design the data structure (as a C struct) for a Library Book Management System. The system must store: book ID, title, author, genre, price, and availability status. Then list the five functions you would declare in library.h, with a one-line description of each.
2. Write a C function `getValidDate(int *day, int *month, int *year)` that reads and validates a date from the user. Day must be 1–31, month 1–12, and year 2000–2099. Use the `fgets + sscanf` pattern and a retry loop for each field.
3. Explain why the variadic `log_event()` function uses `va_list` and `vfprintf` instead of a simple `printf` statement. Write the complete function signature and implementation, and give two examples of how it would be called in an ATM application.
4. For the Employee Payroll Management System, design a complete test case table with at least 6 test cases for the `calculatePaySlip()` function. Include: normal salary, boundary salary (exactly Rs. 21,000 for ESI), salary above Rs. 50,000 for TDS, zero salary, and one invalid input case.
5. A student submits a C project that compiles and runs correctly, but has all code in a single 600-line `main.c` file, uses `scanf` without validation, has no log file, and only one Git commit saying "final project". Identify at least five specific improvements they should make before resubmitting, with a brief justification for each.

Self Assessment — Answer Keys

MCQ

Q.No	Answer	Option Text
1	B	b) Analyse requirements and design module structure
2	C	c) do-while
3	C	c) -1
4	C	c) fgets()
5	B	b) Parse the string buffer into a typed value
6	C	c) strftime()
7	C	c) Append mode preserves existing log entries
8	D	d) <stdarg.h>
9	B	b) Each module/function should have one clearly defined responsibility
10	C	c) Boundary Value
11	B	b) 40% of basic salary
12	B	b) Lists files and patterns that Git should not track
13	C	c) fix
14	B	b) Permanently labels a specific commit as a release version
15	C	c) -g
16	B	b) Project description, build instructions, run instructions, and sample output
17	D	d) ESI

18	C	c) \$@
19	B	b) To ensure all agreed quality criteria are met before delivery
20	C	c) strcspn()

True / False Answer Key

Q.No	Answer	Explanation
1	False	Incorrect — refer to the relevant sub-unit for the correct understanding.
2	True	Correct — refer to the relevant sub-unit for the full explanation.
3	True	Correct — refer to the relevant sub-unit for the full explanation.
4	False	Incorrect — refer to the relevant sub-unit for the correct understanding.
5	True	Correct — refer to the relevant sub-unit for the full explanation.

Short Answer — Model Answers

Q.No	Model Answer
1	Struct: typedef struct { int bookId; char title[80]; char author[60]; char genre[30]; double price; int isAvailable; } Book; — Five functions: addBook(Book arr[], int *count) — adds a new book record; searchBook(const Book arr[], int count, int id) — returns index or -1; issueBook(Book arr[], int count, int id) — sets isAvailable=0; returnBook(Book arr[], int count, int id) — sets isAvailable=1;

	generateCatalogue(const Book arr[], int count) — prints all books to a report file.
2	void getValidDate(int *day, int *month, int *year) { char buf[32]; while(1) { printf("Enter day (1-31): "); fgets(buf,sizeof(buf),stdin); buf[strcspn(buf,"\n")]=0; if(sscanf(buf,"%d",day)==1 && *day>=1 && *day<=31) break; printf("Invalid day.\n"); } — repeat for month (1-12) and year (2000-2099) with appropriate prompts and range checks. }
3	A simple printf cannot accept a format string and variable arguments to pass along to fprintf(fp,...). fprintf() takes a va_list, allowing the caller's format string and arguments to be forwarded to the file. Signature: void log_event(const char *fmt, ...) { FILE *fp=fopen(LOG_FILE,"a"); time_t now=time(NULL); char ts[25]; strftime(ts,sizeof(ts),"%Y-%m-%d %H:%M:%S",localtime(&now)); fprintf(fp,"[%s] ",ts); va_list args; va_start(args,fmt); fprintf(fp,fmt,args); va_end(args); fprintf(fp,"\n"); fclose(fp); } — Usage: log_event("Deposit Rs. %.2f to account %d", amount, accNo); and log_event("Session started.");
4	TC-01: empId=101, basicSalary=20000 → gross=28000, ESI=210, PF=2400, TDS=0, net=25390 (PASS). TC-02: basicSalary=15000 → gross=21000 (ESI boundary), ESI=157.50 (PASS). TC-03: basicSalary=35714 → gross=50000 (TDS boundary), TDS=0 (PASS/edge). TC-04: basicSalary=36000 → gross=50400, TDS=5040 (>50000 triggers TDS, PASS). TC-05: basicSalary=0 → all components=0, netSalary=0 (PASS). TC-06: basicSalary=-5000 → should return error/reject (validation gate).
5	Five improvements: (1) Decompose into modules — employee.c, payroll.c, logger.c with header files; single 600-line main.c violates SRP and is unmaintainable. (2) Add input validation — replace all scanf with fgets+sscanf+range check; unvalidated input causes crashes and security

issues. (3) Add a logging system — implement `log_event()`; without logs there is no audit trail for submitted work or production bugs. (4) Improve Git history — at least 8 meaningful commits with conventional messages (feat/fix/chore); one "final project" commit shows no development discipline and makes code review impossible. (5) Add a test case table — document normal, boundary, and error test cases with expected vs actual output; no tests means no evidence of correctness.

ONLINE DEGREE PROGRAMS OFFERED

B.Com | BBA | MBA | MCA | M.Com | MA

